thing like movies[:mcu][1][:actors][1][:first_name]. Not only is the access complicated, but validating that the data has the shape you are looking for can also be difficult.

In Ruby, *pattern matching* is designed to make both these tasks easier by allowing you to specify the structure of the data as a pattern, and assign values to the parts of the data that match. Please note that many programming languages have things they call pattern matching, but Ruby's implementation is somewhat different from many of these. The most similar seems to be in Python.

Pattern matching in Ruby is fully available in versions 3.0 and up (version 2.7 had an experimental version), and some of the single-line features were added in version 3.1

Pattern matching in Ruby compares a *target*, which can be any Ruby object, to a *pattern*. A pattern is also a Ruby object, but the pattern can also contain the names of not-yet-defined local variables. If the target matches the pattern, the target is deconstructed into the pattern, setting the value of those variables.

Single-Line Pattern Matching

Ruby uses the keyword in to match a target to a pattern. When used this way, the expression returns true if the target matches, and false if it doesn't.

The simplest pattern match is that values match themselves:

```
"banana" in "banana" # => true
"banana" in "apple"  # => false
3 in 3               # => true
3 in 5               # => false
2i in 2i             # => true
```

Behind the scenes, the pattern match for values uses the same === triple-equal operator that case statements do. This means that classes that implement === can do more general matches. As we saw earlier, a class is === to instances of that class, regular expressions are === to strings that match the expression, and ranges are === to elements covered by the range.

So we can write the following patterns:

```
"banana" in String   # => true
"banana" in Integer  # => false
"banana" in /b(an+)/ # => true
3 in 1..10           # => true
```

That's starting to look a little more interesting. The next step is that you can match not just scalar values, but arrays and hashes. Each subelement of the array or hash can be a pattern that matches the associated element in the target. For an array, every element in the array must match the associated pattern element. For a hash, only keys in the pattern must match, the existence of other keys in the target does not fail the match. If you want to have an exact hash match, you need to include **nil in the pattern.

```
[1, 2, 3] in [Integer, Integer, Integer]          # => true
[3, "banana", "apple"] in [1..10, String, /p{2}/] # => true
{name: "Fred", city: "Bedrock"} in {city: String} # => true
{name: "Fred", city: "Bedrock"} in {}             # => false
```